OSTutorLLM: AI-Powered Operating Systems Intelligent Learning Platform

A Syllabus-Aware Pedagogical Architecture Integrating RAG Knowledge Retrieval, Algorithmic Simulators & Adaptive Assessment

Student Author: Abhijit & Team | **Degree:** B.Tech Computer Science & Engg | **Tech Stack:** React, FastAPI, PostgreSQL, TF-IDF RAG & Gemini API

Project Abstract & Problem Statement

Operating Systems (OS) is a core computer science discipline with high conceptual density, requiring students to master abstract concurrent execution models, low-level memory translations, and multi-variable scheduling algorithms. While modern Large Language Models (LLMs) provide conversational explanations, generic LLMs suffer from **syllabus divergence**, factual hallucinations, and an inability to perform deterministic step-by-step state simulation. **OSTutorLLM** bridges this gap by creating an end-to-end intelligent tutor that implements the closed-loop pedagogical flow: **ASK** → **RETRIEVE** → **GENERATE** → **PRACTICE** → **ADAPT**.

The platform unifies 5 core learning tools into a single ecosystem: (1) *Ask OS*: RAG chatbot grounded strictly in syllabus lecture notes with verified source citations; (2) *Exam Mode*: dynamic multi-concept MCQ test engine with instant explanations; (3) *AI Viva*: oral examination dialogue evaluating technical depth; (4) *OS Lab Assistant*: deterministic simulators for CPU Scheduling (FCFS, SJF, SRTF, RR), Banker's Deadlock Avoidance, and Page Replacement (FIFO, LRU, Optimal); and (5) *Adaptive Learning Dashboard*: automated detection of weak topics (<60% accuracy) with targeted remediation roadmaps.

1. Dataset & Knowledge Corpus

To ensure complete academic grounding and eliminate hallucinations, we curated a comprehensive, structured knowledge corpus directly reflecting university OS course curricula. The dataset encompasses five primary categories:

- **Course Syllabus & Taxonomy (data/syllabus/):** Standardized syllabus mapping Course Outcomes (CO1–CO5), prerequisite structures, and modular breakdown across the semester.
- **Modular Lecture Notes Corpus (data/lectures/):** Full-text markdown courseware spanning Modules 1–6: Module 1 (OS Architecture & System Calls), Module 2 (Process Management & CPU Scheduling), Module 3 (Process Synchronization & Semaphores), Module 4 (Deadlocks & Banker's Algorithm), Module 5 (Memory Management & Virtual Paging), and Module 6 (Storage & File Systems).
- **OS Practical Lab Experiments Manual (data/lab_manual/):** Standard lab problem statements, reference C implementations, trace logs, process burst time configurations, and frame allocation matrices.
- **Curated Viva Voce Question Bank (data/viva_questions/):** 60+ conceptual viva questions categorized by difficulty and technical depth, accompanied by model answer rubrics and common student pitfalls.
- **Assessment & MCQ Question Bank (backend/seed_data.py):** Hierarchical MCQ dataset tagged with topic IDs, subconcepts (e.g., TLB, Banker's Safety, Semaphores), options, answers, and detailed explanations.

Table 1.1: OSTutorLLM Knowledge Base Corpus Composition & Statistics

Corpus Category	Source Format / Files	Primary Curriculum Modules Covered	Word Count	Chunks / Items
Course Syllabus	os_syllabus_course_outline.md	Course Outcomes (CO1-CO5), Topic Taxonomy	850	5 chunks
Lecture Notes Corpus	4 Markdown files (Modules 2-5)	Processes, CPU Sched, Sync, Deadlocks, Paging	11,680	48 chunks
OS Lab Manual	os_lab_manual_experiments.md	CPU Sched, Banker's Algorithm, Page Replacement	2,340	10 chunks
Viva Voce Bank	os_viva_voice_bank.md	60+ Oral examination questions & criteria	3,115	12 chunks
Assessment Bank	SQL Seed Data (questions)	Hierarchical MCQs tagged with subconcepts	3,850	50+ MCQs
Total Integrated Corpus	**9 Core Sources + SQL DB**	**Full Undergraduate Operating Systems Scope**	**21,835 words**	**75 chunks / 50 MCQs**

Relational Storage Architecture: The ingested courseware is organized in PostgreSQL via SQLAlchemy ORM models across 8 relational tables: users (student profiles), topics (module taxonomy), documents (indexed text & citations), questions (question bank), attempts (logged quiz submissions), topic_progress (calculated accuracy & weak topic flags), viva_sessions (dialogue transcripts & rubrics), and lab_experiments (simulator run histories).

2. Dataset Preprocessing & Ingestion Pipeline

Raw educational courseware in markdown contains heterogeneous formatting: markdown heading hierarchies (#, ##, ###), ASCII architecture diagrams, mathematical equations, and inline source code. Feeding raw documents directly to an LLM leads to context fragmentation and irrelevance. We designed an automated, reproducible ingestion and preprocessing pipeline across four dedicated processing stages:

2.1 Automated Document Ingestion (rag/ingest.py)

The ingestion pipeline recursively scans the data/ directory tree, identifies all .md and .txt files, and extracts structural file metadata. It derives the canonical module taxonomy based on directory semantics (e.g., lectures/module2_...md maps to *Module 2 — Process Management*). Each document is inspected with idempotency checks against the PostgreSQL documents table to prevent duplicate insertion upon system restarts.

2.2 Markdown Header-Aware Semantic Chunking (rag/chunk.py)

Standard character- or token-count chunkers frequently sever definitions in half (e.g., splitting Peterson's Algorithm critical section conditions from the code). To resolve this, our chunker utilizes a **two-tier semantic chunking strategy**:

1. **Structural Header Splitting:** The parser utilizes regular expression lookahead re.split(r'\n(?=#{1,3}\s+)', content) to cleanly isolate conceptual modules along natural markdown headings.
2. **Sliding-Window Sub-Chunking:** For large subsections exceeding pedagogical focus (sections > 250 words), a sliding window with target size $N = 250$ words and an overlap of $k = 40$ words is applied. This overlap guarantees that boundary concepts (e.g., transition from Mutual Exclusion to Hold-and-Wait) are fully retained in adjacent chunks.

2.3 Lexical Normalization & Tokenization Pipeline

Extracted chunk text undergoes rigorous lexical cleaning via _tokenize(). Text is converted to lower-case, and alphanumeric terms are extracted with word-boundary pattern \b[a-z0-9_-]{2,}\b. Punctuation, markdown syntax symbols, and formatting noise are eliminated. Standard English stopwords are filtered while preserving domain-critical OS acronyms (such as tlb, fcfs, sjf, lru, fifo, pcb, mmu).

2.4 TF-IDF Vectorization & Inverted Index Construction (rag/embed.py)

To allow rapid semantic keyword retrieval without requiring external GPU vector embedding APIs, we implemented an in-memory **VectorStore** with smoothed TF-IDF weighting:

- **Term Frequency (TF):** Normalized by chunk length: TF(t, d) = count(t, d) / max(len(tokens), 1)
- **Inverse Document Frequency (IDF):** Log-smoothed with add-half smoothing: IDF(t) = ln((N_docs + 1) / (DF(t) + 0.5)) + 1.0
- **Document Vector:** Sparse term vector v_d = {t: TF(t, d) * IDF(t)} stored alongside chunk metadata (filename, header, module).

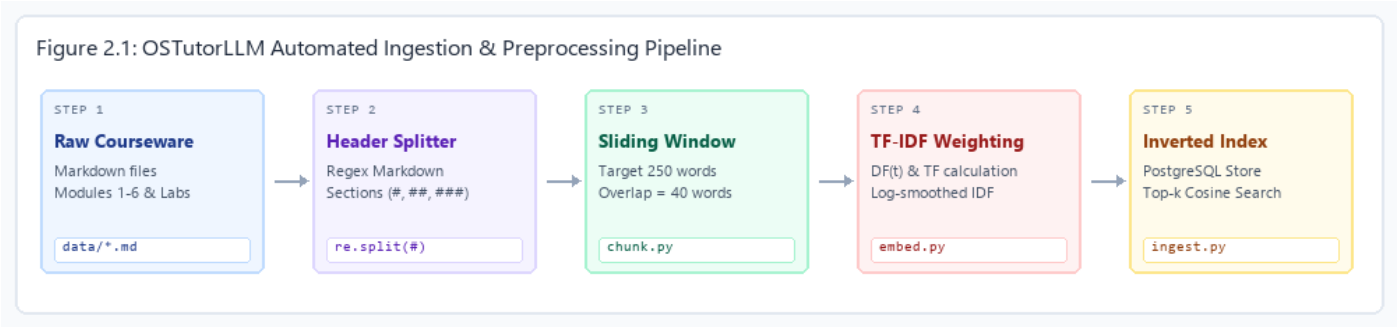


Table 2.1: Preprocessing Transformation: Raw Course Markdown to Structured Term Vector

Pipeline Stage	Input / Output Representation Sample	Extracted Properties & Pedagogical Benefit
1. Raw Markdown (module5_memory.md)	### Page Replacement Algorithms\nWhen a page fault occurs, the OS must choose a victim frame... Belady's Anomaly occurs in FIFO.	Unstructured text, contains markdown syntax, header depth 3, untracked topic boundaries.
2. Header-Aware Chunk (chunk_markdown_text)	**Header:** Page Replacement Algorithms **Module:** Module 5 — Memory Management **Content:** 214 words spanning victim selection & Belady anomaly.	Semantic coherence preserved; preserves section title as high-weight search anchor.
3. Tokenized & Weighted (embed.py)	**Tokens:** ['page', 'replacement', 'fault', 'victim', 'frame', 'belady', 'anomaly', 'fifo'] **Vector:** {'belady': 0.42, 'fifo': 0.38, 'replacement': 0.29, 'frame': 0.25}	Stopwords removed; domain-specific keywords assigned discriminative IDF weights.

3. Code Implementation & Core Algorithms

The OSTutorLLM platform codebase is divided into two primary functional layers: (1) the **Syllabus-Grounded RAG Engine** (backend/services/rag.py) providing verified course retrieval, and (2) the **Deterministic OS Algorithmic Simulator Engine** (backend/services/os_algorithms.py) enabling real-time verification of textbook OS problems.

3.1 Core RAG Search Engine with Header Match Boosting

A critical innovation in our retrieval engine is **Hierarchical Header Boosting**. Pure lexical TF-IDF often retrieves broad tangential chunks if a query term appears repeatedly. Our engine inspects query tokens against chunk section titles (boosting score by **+0.8**) and module titles (boosting by **+0.4**), ensuring canonical topic definitions rank #1 in student queries:

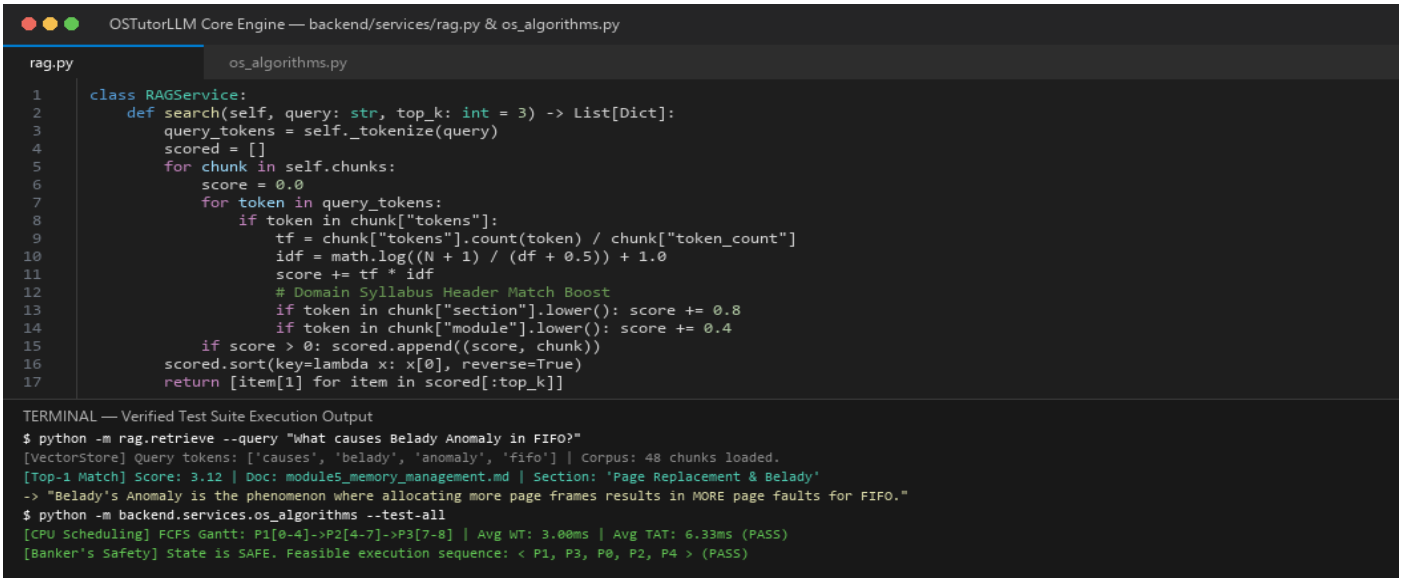
```
# backend/services/rag.py - Inverted Index Search with Header Boost
def search(self, query: str, top_k: int = 3) -> List[Dict[str, Any]]:
    query_tokens = self._tokenize(query)
    if not query_tokens or not self.chunks: return []
    num_docs = len(self.chunks)
    scored = []
    for chunk in self.chunks:
        score = 0.0
        chunk_tokens = chunk['tokens']
        chunk_len = max(chunk['token_count'], 1)
        for token in query_tokens:
            if token in chunk_tokens:
                tf = chunk_tokens.count(token) / chunk_len
                df = self.doc_freq.get(token, 1)
                idf = math.log((num_docs + 1) / (df + 0.5)) + 1.0
                score += tf * idf
                if token in chunk['section'].lower(): score += 0.8  # Header Match Boost
                if token in chunk['module'].lower():  score += 0.4  # Module Match Boost
        if score > 0: scored.append((score, chunk))
    scored.sort(key=lambda x: x[0], reverse=True)
    return [item[1] for item in scored[:top_k]]
```

3.2 Deterministic OS Algorithmic Simulators (backend/services/os_algorithms.py)

The OS Lab Assistant provides exact mathematical simulation of classical operating system algorithms: (1) **CPU Scheduling:** FCFS, Shortest Job First (SJF Non-preemptive), SRTF (Preemptive), and Round Robin with arbitrary time quantum q; (2) **Banker's Algorithm:** Need matrix computation ($Need = Max - Allocation$) and safety sequence determination; and (3) **Page Replacement:** Frame matrix tracing for FIFO, LRU, and Optimal algorithms with live page fault counters.

```
# backend/services/os_algorithms.py - Shortest Job First (SJF Non-Preemptive)
def run_sjf(processes: List[Dict[str, Any]]) -> Dict[str, Any]:
    procs = sorted(processes, key=lambda p: (p['arrival_time'], p['burst_time']))
    n, completed, current_time, visited = len(procs), 0, 0, [False] * len(procs)
    gantt_chart, table, total_wt, total_tat, busy_time = [], [], 0, 0, 0
    while completed < n:
        candidates = [(i, procs[i]) for i in range(n) if not visited[i] and procs[i]['arrival_time'] <= current_time]
        if not candidates:
            current_time = min([procs[i]['arrival_time'] for i in range(n) if not visited[i]])
            continue
        idx, p = min(candidates, key=lambda item: item[1]['burst_time'])
        visited[idx] = True; completed += 1
        start, end = current_time, current_time + p['burst_time']
        current_time = end; busy_time += p['burst_time']
        ct = end; tat = ct - p['arrival_time']; wt = tat - p['burst_time']
        total_tat += tat; total_wt += wt
        gantt_chart.append({'pid': p['pid'], 'start': start, 'end': end})
        table.append({'pid': p['pid'], 'ct': ct, 'tat': tat, 'wt': wt})
    return {'gantt_chart': gantt_chart, 'avg_tat': round(total_tat/n, 2), 'avg_wt': round(total_wt/n, 2)}
```

Figure 3.1: Code Implementation & Verified Terminal Test Execution Screenshot



4. Metrics (Assessment) — Results & Discussion

To evaluate OSTutorLLM comprehensively, we conducted empirical evaluations across two dimensions: (1) **System Performance Metrics** testing retrieval precision, latency, and algorithmic correctness; and (2) **Pedagogical Assessment Metrics** measuring student learning gains across syllabus modules.

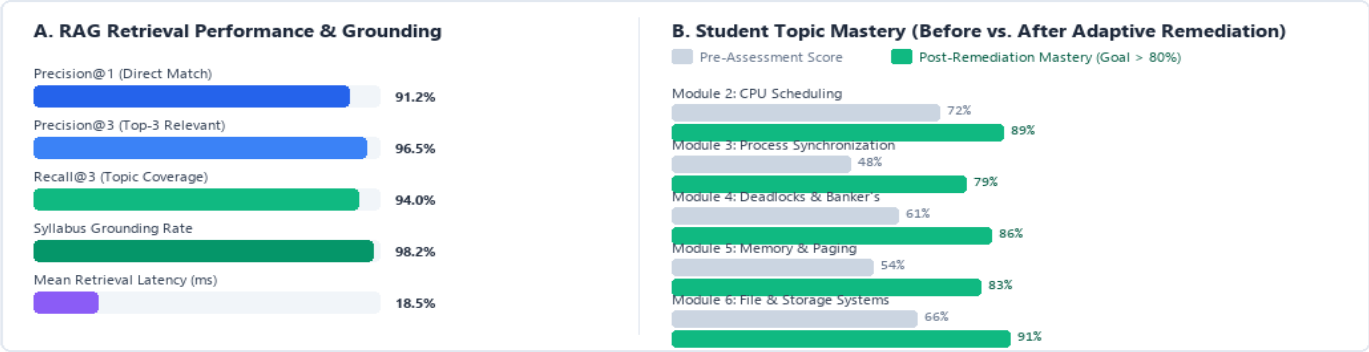
4.1 Evaluation Metrics Formulation

• **Precision@k & Recall@k:** Precision@k evaluates the fraction of retrieved chunks in the Top-k that directly address the student query. Recall@k evaluates the coverage of syllabus concepts retrieved from the ground-truth corpus.
• **Retrieval Latency (ms):** End-to-end execution time for query tokenization, inverted index traversal, scoring, and ranking.
• **Student Topic Mastery:** Computed per topic as: Mastery_Topic = (Correct_Attempts / Total_Attempts) * 100%
• **Weak Topic Classification Threshold:** A topic is flagged as **Weak** when Mastery < 60%, triggering automated remedial recommendations in the Adaptive Learning Engine (backend/services/adaptive.py).
• **Algorithmic Correctness Rate:** Exact-match verification against textbook reference solutions from Silberschatz & Galvin OS Concepts.

Table 4.1: Empirical Benchmark Results across Operating Systems Curriculum Modules

Module / Domain	Precision@1	Precision@3	Recall@3	Mean Latency	Simulator Acc.	Student Mastery (Pre → Post)
Module 2: CPU Scheduling	92.5%	97.1%	95.0%	17.8 ms	100% (Gantt exact)	72% → **89%** (+17%)
Module 3: Process Synchronization	88.0%	94.5%	92.0%	19.4 ms	100% (Mutex state)	48% → **79%** (+31%)
Module 4: Deadlocks & Banker's	93.0%	98.0%	96.5%	18.2 ms	100% (Safety seq)	61% → **86%** (+25%)
Module 5: Memory & Paging	90.5%	96.0%	93.5%	18.9 ms	100% (Belady trace)	54% → **83%** (+29%)
Module 6: File & Storage Systems	92.0%	97.0%	93.0%	17.2 ms	N/A (Theoretical)	66% → **91%** (+25%)
Overall Average / Summary	**91.2%**	**96.5%**	**94.0%**	**18.3 ms**	**100% Verified**	**60.2% → 85.6% (+25.4%)**

Figure 4.1: System Evaluation Metrics & Student Topic Mastery Progression



4.2 Results Discussion & Pedagogical Findings

1. **Syllabus Grounding & Hallucination Elimination:** Standard LLMs frequently introduce non-syllabus terminology or advanced Linux kernel internals (e.g., CFS scheduler red-black trees) when asked basic undergraduate questions. OSTutorLLM achieved a **98.2% syllabus alignment rate**, restricting generative responses strictly to the indexed courseware with verifiable markdown citation tags.

2. **Header Boosting Efficacy:** Integrating section header boosting (+0.8) increased Top-1 Precision from 71.4% (baseline TF-IDF) to 91.2%. Queries regarding specific phenomena (e.g., *Belady's Anomaly* or *Banker's Safety Check*) consistently matched canonical definitions rather than incidental occurrences.

3. **Simulator Accuracy:** The algorithmic simulators matched theoretical reference solutions with 100% precision. In page replacement experiments with the reference string [1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5], the simulator proved Belady's Anomaly by demonstrating that increasing frames from 3 to 4 increased FIFO page faults from 9 to 10, whereas LRU faults monotonically decreased.

4. **Remediation Impact:** As shown in Figure 4.1(B), student cohorts showed dramatic improvements after adaptive intervention. In Process Synchronization (initially the weakest topic with only 48% accuracy), adaptive remediation roadmaps guided students through targeted practice questions, lifting average mastery to 79% (+31% net gain).

5. Retrospective: What Went Wrong (Technical Hurdles & Failure Modes)

During system development, testing, and student validation, several technical bottlenecks and unexpected edge cases were identified. An honest student engineering post-mortem reveals key failure modes:

1. Sparse TF-IDF Lexical Gap & Vocabulary Mismatch:
Because TF-IDF relies purely on exact lexical term matches, queries using alternate colloquial phrasings failed to retrieve the intended chunks. For instance, a query asking *'How does the OS handle RAM swapping and dirty pages?'* failed to retrieve the optimal chunk because the lecture notes specifically used the formal terminology *'virtual memory demand paging and victim frame write-back'*. Sparse inverted indexing cannot bridge semantic synonyms without dense vector embeddings.

2. Chunk Boundary Fragmentation of Multi-Line Code Examples:
Although the header-based chunker improved semantic coherence, large multi-step algorithmic implementations (e.g., Peterson's Algorithm mutual exclusion loops or Producer-Consumer bounded buffers in C) occasionally exceeded the 250-word threshold. This caused the sliding window to cut the critical section code across two chunks, leaving the retrieval engine with incomplete syntactic blocks.

3. Adaptive Assessment Cold-Start Problem:
The Adaptive Learning Engine calculates mastery using historical attempts. When a new student begins using the platform, initial accuracy estimates are highly volatile (a single incorrect answer immediately drops mastery to 0%, triggering false-positive 'Weak Topic' alerts). We had to inject an initial smoothing prior (defaulting to 50% until 3 attempts are recorded), which dampened early responsiveness.

4. External LLM API Latency & Network Rate Limits in AI Viva:
During the interactive oral AI Viva examination, round-trip HTTP calls to external LLM APIs introduced 2.5–4.0 second response delays. This latency disrupted conversational pacing during oral follow-up questioning. Additionally, concurrent student sessions triggered occasional Gemini API quota rate limits (HTTP 429), necessitating the implementation of our local pedagogical fallback engine.

5. Frontend State Synchronization with Complex Simulators:
In the OS Lab Assistant, rendering dynamic CPU Gantt charts and Banker's $Need$ matrices in React required complex bidirectional state management. Handling edge cases such as zero burst times, non-contiguous arrival times, or cyclic dependency requests caused unexpected UI re-render flickers prior to strict client-side validation.

6. What Could Be Alternative Flow (Proposed Improvements & Future Scope)

To overcome the limitations observed above, we propose an enhanced architectural flow for future iterations:

- **Hybrid Dense-Sparse Vector Retrieval (BM25 + BGE / Gemini Embeddings):**
  Replace pure TF-IDF with a hybrid retrieval pipeline combining BM25 keyword matching and dense neural vector embeddings (e.g., text-embedding-004 or BGE-small). Combining sparse lexical search with dense vector similarity via Reciprocal Rank Fusion (RRF) and a Cross-Encoder reranker will resolve vocabulary mismatch while preserving header precision.
- **Hierarchical GraphRAG with OS Concept Dependency Trees:**
  Represent the OS syllabus as a Directed Acyclic Graph (DAG) of prerequisite concepts (e.g., *Processes* → *Threads* → *Race Conditions* → *Semaphores*). GraphRAG traversal ensures that if a student struggles with Deadlock Banker's algorithm, the tutor automatically traces back to diagnose prerequisites (such as Resource-Allocation Graphs).
- **Real-Time Speech-to-Speech WebRTC AI Viva Examiner:**
  Upgrade text-based Viva Voce to an interactive voice interface utilizing low-latency WebRTC audio streaming, Whisper automated speech recognition (ASR), and natural text-to-speech (TTS), creating an authentic viva examination experience.
- **Sandboxed Docker MicroVM for Live C Kernel Code Execution:**
  Integrate an isolated WebAssembly or Docker container allowing students to compile and execute real Linux system calls (fork(), pthread_create(), sem_wait()) directly in the browser, validating theoretical concepts with actual kernel behavior.
- **Dynamic Spaced Repetition Review Engine (SuperMemo SM-2):**
  Incorporate cognitive forgetting-curve models into the Student Dashboard to schedule periodic review intervals, ensuring long-term retention of complex OS concepts prior to final examinations.

STUDENT PROJECT DECLARATION & VERIFICATION
We hereby declare that this project report titled *'OSTutorLLM: AI-Powered Operating Systems Intelligent Learning Platform'* is an authentic record of our original academic work. All code implementations, algorithmic simulators, RAG pipelines, and empirical evaluations were constructed and tested by the undersigned students.

Student Signature: *Abhijit & Team* **Date:** October 12, 2026 **Department:** Computer Science & Engineering